

SLP: A Secure and Lightweight Scheme Against Content Poisoning Attacks in Named Data Networking Based on Probing

Kunpeng Ding[✉], Jiayu Yang, *Member, IEEE*, Kaiping Xue[✉], *Senior Member, IEEE*,
Jiangping Han[✉], *Member, IEEE*, Jian Li[✉], *Senior Member, IEEE*,
Qibin Sun[✉], *Fellow, IEEE*, and Jun Lu[✉]

Abstract—Named Data Networking (NDN) stands out as a promising Information Centric Networking architecture capable of facilitating large-scale content distribution through in-network caching and location-independent data access. However, attackers can easily inject poisoned content into the network, called content poisoning attacks, which leads to a substantial deterioration in user experience and transmission efficiency. In existing schemes, routers fail to determine the contamination source of received poisoned content, leading to the inability to accurately identify attacker nodes. Besides, attackers' dynamic behaviors and network instability could disrupt identification results. In this paper, we propose a Secure and Lightweight scheme against content poisoning attacks based on Probing (SLP), where a proactive and reliable probing protocol is designed to identify adversaries quickly and precisely. In SLP, a router sends specifically chosen interest packets to probe a suspicious node, so that the returned corresponding content can straightly reflect its trustworthiness without other nodes' interference. In addition, a hypothesis testing algorithm is developed to analyze the returned content, which can exclude the impact of transmission errors and adapt to dynamic attackers. Moreover, we utilize users' feedback to avoid unnecessary probing costs on unaffected routers, with its reliability guaranteed by an efficient cuckoo-filter-based feedback validation mechanism. Security analysis shows that SLP achieves resistance against content poisoning attacks and malicious feedback. The experimental results demonstrate that SLP makes users hardly be affected by attacks and brings in only slight overhead.

Index Terms—Named data networking, content poisoning attack, probe, feedback, hypothesis testing.

I. INTRODUCTION

NAMED Data Networking (NDN) [1], proposed as a novel network paradigm, shifts the Internet architecture from a host-centric model to a content-centric model. In NDN, users request desired content by sending an interest packet

specifying the content name. When the interest packet is forwarded based on this name to a content producer that owns the content or a router that caches the content, a data packet carrying the content will be sent back to the requesting user along the interest packet routing path reversely. With the characteristics of in-network caching and location-independent data access pattern, NDN can optimize bandwidth utilization and shorten content retrieval time.

Despite these advantages, NDN poses new security challenges [2], [3], [4], [5], [6], [7]. The content poisoning attack is a typical one that undermines data integrity, where an adversary injects fake content bearing legitimate names into the network, by masquerading as a legitimate content producer or compromising a vulnerable router. Although NDN enables each data packet to carry a digital signature that can be verified by routers and users, routers cannot afford to verify signatures of each data packet at line speed due to the huge verification overhead [8], [9]. Therefore, in the absence of routers' verification, fake content can easily reach users, who will verify it and retransmit a same interest packet to retrieve authentic content. In consequence, users' interest satisfaction ratio decreases and content retrieval delay increases significantly. What's worse, on the transmission path where the fake content is delivered to users, it will be cached in intermediate routers if not verified, which will be used to serve subsequent interest packets, leading to a wider range of users isolated from authentic content and caches contaminated.

Researchers have proposed solutions categorized into content-oriented schemes, e.g., [10], [11], [12], [13], [14], [15], [16], [17], [18], and [19] and node-oriented schemes, e.g., [20], [21], [22], [23], [24], [25], [26], [27], [28], [29]. Content-oriented schemes aim to enhance the efficiency of verifying all content by utilizing lightweight signature algorithms or adopting selective verification methods. However, in the network where routers may be compromised, the lack of security makes these schemes fail to identify all fake content. In addition, even if routers discard all fake content, users are not likely to obtain authentic content in time as attackers persistently inject fake content. On the other hand, node-oriented schemes focus on identifying attacker nodes, where routers derive trust values of neighbor nodes from verification results and accordingly choose secure forwarding paths. However, there is a risk of honest routers inadvertently transmitting fake content by forwarding unverified received content or responding to

Manuscript received 25 December 2023; revised 14 July 2024; accepted 20 August 2024; approved by IEEE/ACM TRANSACTIONS ON NETWORKING Editor P. P. C. Lee. Date of publication 5 September 2024; date of current version 19 December 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 61972371, Grant 62372425, and Grant 62302472; and in part by the Youth Innovation Promotion Association of Chinese Academy of Sciences (CAS) under Grant Y202093. (*Corresponding author: Kaiping Xue.*)

Kunpeng Ding, Jiayu Yang, Kaiping Xue, Jiangping Han, Jian Li, and Qibin Sun are with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei, Anhui 230027, China (e-mail: kpxue@ustc.edu.cn).

Jun Lu is with the Department of Electronic Engineering and Information Science and the School of Cyber Science and Technology, University of Science and Technology of China, Hefei, Anhui 230027, China.

Digital Object Identifier 10.1109/TNET.2024.3451231

1558-2566 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

interest packets with contaminated caching. Consequently, the trust value is inaccurate and misleading, excluding many honest routers from the transmission path.

Our key idea is to detect attackers with utmost precision. There are three challenges in the design. The first challenge is what criterion should be chosen to determine if a node is trustworthy. We observe that when a node receives fake content from its neighbor node, it only indicates that some upstream router or producer tampered with that content, but it is uncertain whether that neighbor node is the attacker. Hence, the authenticity of passively received data packets is an unreliable criterion since these packets usually traverse many routers and can possibly be modified by any of them. Instead, we choose the authenticity of those data packets that is only related to one node as our criterion. Considering that the transmission path of data packets is decided by interest packets, we send specifically chosen interest packets to obtain such data packets, called proactive probing.

The second challenge is how to correlate the authenticity of returned data packets to the trustworthiness of the neighbor node in face of dynamic attackers and unstable network. It is inaccurate to rely solely on a single probing packet, since it is possible to receive authentic content from a malicious node who is attacking probabilistically and it is also possible to receive fake content from an honest node due to transmission errors. Here we develop a hypothesis testing algorithm to make correct decisions, which determines the suitable quantity of probing packets to send and conducts a formal mathematical analysis of the received data packets.

The third challenge is determining the timing and nodes for sending probing packets. Constantly probing every node leads to wastage of computation and bandwidth resources. Instead, users' feedback, which is a retransmitted same interest packet upon receiving fake content, can be used to ascertain both the timing and nodes for probing. However, relying on users' feedback introduces new security risks due to its lack of trustworthiness, so we design a lightweight cuckoo-filter-based validation mechanism to identify malicious feedback.

Therefore, in this paper, we propose SLP, a secure and lightweight scheme against content poisoning attacks based on probing, in which proactive probing substitutes for passive receiving to make verification results of data packets only related to one node rather than interfered with by other nodes. In addition, to adapt to dynamic attackers and unstable network, we develop a hypothesis testing algorithm to decide how many probing packets to send and how to obtain accurate trustworthiness of the neighbor node from the returned data packets. Moreover, we utilize users' feedback to determine the optimal probing timing and nodes, avoiding unnecessary probing overhead. Meanwhile we design a cuckoo-filter-based validation mechanism to distinguish malicious feedback at low costs. Our contributions can be summarized as follows:

- We propose a proactive probing scheme to identify attackers accurately and quickly. The form and payload of probing packets are specifically designed to eliminate interference from irrelevant nodes.
- We combine a pair of interrelated hypothesis testing to control all kinds of errors, which conducts rigorous

mathematical analysis of the probing results to adapt to dynamic attackers and unstable networks.

- We design a lightweight cuckoo-filter-based validation mechanism to identify malicious feedback constructed in various ways and derive probing timing and nodes from reliable feedback, thus avoiding unnecessary probing overhead during non-attack time or on unaffected routers.
- We formally analyze the security of our scheme, and prove that it can defend against content poisoning attacks even in the network where all entities are untrusted. Experimental results show that our scheme can make the network recover from the attack quickly and users hardly affected, while bringing in only slight overhead.

The rest of this paper is organized as follows. Section II introduces the related work. Section III shows the preliminaries. In section IV, we present the system model, security assumption, and design goals. The proposed scheme is given in section V. Then section VI and section VII show the security and performance analysis respectively. Finally we conclude this paper in section VIII.

II. RELATED WORK

The content poisoning attack is implemented by either a malicious content producer or a compromised router. Mitigating schemes can be divided into two categories: content-oriented schemes, and node-oriented schemes.

Content-oriented schemes consider how to verify content efficiently. In [10] and [11], the authors adopted the self-certifying naming mechanism, where the hash of the content or the public key is appended to the name to make verification simpler. In [12], a one-time signature algorithm is proposed to avoid the heavy verification costs of traditional digital signatures. In [13], verification is done only when content is hit in the caching, and the verified cached content is moved to the protected region of caching. Li et al. [14] constructed a trust chain that combines authority-based trust and neighbor-based trust, where certificates are collected hop by hop to update the trust chain. In [15], cached content is ranked by users' feedback, and the highest-ranked content is chosen to respond to a request. Huang et al. [16] adopted a certificateless group signature algorithm and a batch verification scheme to speed up the verification process. In [17], a certificateless signcryption scheme is applied, where encryption and signing are performed logically in a single step. Xue et al. [18] proposed a collaborative verification protocol, which leverages bloom filters to store and share historical verification results, and uses a probabilistic verification formula to make content verified only once. In [30], a new trust schema API is developed, which addresses the need for comprehensive trust schema functionalities, an easy-to-use schema language, and an extensible and flexible design. In [31], a distributed capability-based security enforcement architecture is proposed, where CP generates capabilities for data packets and routers maintain the request token set and the response token set that are updated each round to verify the interest and data packet efficiently. In [19], content producers' reputation is measured by edge routers, which is used to dynamically decide proper parameters in probabilistic batch verification of data packets

to reduce costs dramatically. In the schemes above, either the overhead is not affordable, or security and effectiveness can be undermined by compromised routers.

Node-oriented schemes consider how to identify whether a node is an attacker. In [20], the next hop that returned fake content becomes the least preferred option to forward interest packets, leading to the inhibition of malicious producers. In [21], each router shares a key with every node k hops away, and each data packet carries cryptographic MACs needed to check the validity of previous k hops. Cui et al. [22] leveraged users' feedback to trace evil producers and adjust forwarding paths. In [23], routers compute the trust value of each content and neighbor node based on users' feedback, and accordingly decide whether to forward that content and whether to receive interest packets from that neighbor node. Nguyen et al. [24] proposed a set of 18 metrics to feature the NDN data plane and designed micro detectors to raise alarms when a metric strays from its expected behavior, which are used in the Bayesian network to infer anomalies. In [25], probabilistic verification and neighbor routers' recommendations are aggregated in the kalman filter to rank neighbor nodes. Saha et al. [26] proposed a blacklist method, where every node records the number of genuine and fake contents received from another node, and interacts with each other to blacklist malicious nodes. Kar et al. [27] applied a Stackelberg game-theoretical model to simulate the attacker-defender situation and used the hop count between the source and destination to generate the public key. Anisetti et al. [28] presented a certification methodology to model the expected behavior in the network, where nonfunctional properties are verified over time to capture changes in the security status of the corresponding status. In [29], edge routers evaluate content producers and assign tokens periodically to find the malicious producer, and core routers reach a consensus to find the compromised core router using the blockchain. Meanwhile, each router adopts the policy in [13], incorporating a popularity-based probabilistic caching and caching replacement method. The schemes above cannot rapidly identify attackers without affecting honest nodes. Besides, some of them are impractical and those reliant on users' feedback fail to guarantee its authenticity efficiently.

III. PRELIMINARIES

A. Cuckoo Filter

The cuckoo filter [32] is used for approximate membership tests with very good space efficiency and low computation costs, which is described as follows:

- **Set up:** Generate (cf, f, h, k) , where cf is an empty array of m buckets with each bucket having b entries of l bits, f is a hash function to compute the fingerprint of an item, h is a hash function to compute the index of an item.
- **Insert:** For an item x , compute its fingerprint $fin = f(x)$, indexes $i_1 = h(x)$ and $i_2 = i_1 \oplus h(fin)$. If the bucket at index i_1 or i_2 has an empty entry, insert fin to that entry. Otherwise, insert fin to a randomly selected entry at index i_1 or i_2 , and find an alternative location for the original item in that entry.
- **Search:** For an item x , compute its fingerprint $fin = f(x)$, indexes $i_1 = h(x)$ and $i_2 = i_1 \oplus h(fin)$. Return

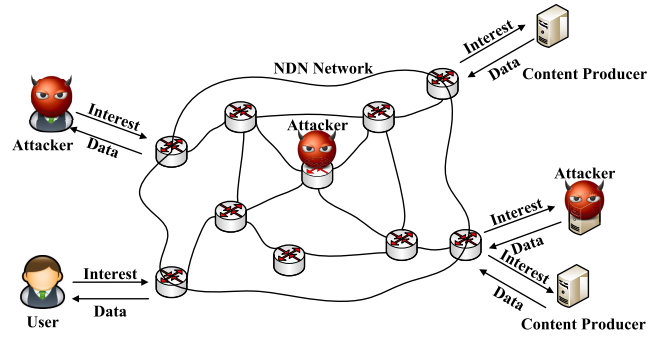


Fig. 1. System model.

true if the bucket at index i_1 or i_2 has fin , and return false otherwise.

- **Delete:** For an item x , compute its fingerprint $fin = f(x)$, indexes $i_1 = h(x)$ and $i_2 = i_1 \oplus h(fin)$. If the bucket at index i_1 or i_2 has fin , remove it.

B. Hypothesis Testing

Hypothesis testing [33] is a statistical technique used to assess whether observed data provides enough evidence to support or reject a stated hypothesis about a population parameter, which is described as follows:

- **Formulate Hypotheses:** Assume that the population is from a parameter family of distributions $\{F(x, \theta), \theta \in \Theta\}$, where θ is the unknown parameter, Θ is the parameter space. We have two hypotheses about whether θ belongs to a parameter subspace Θ_0 : Null Hypothesis (H_0): $\theta \in \Theta_0$; Alternative Hypothesis (H_1): $\theta \notin \Theta_0$.
- **Collect Data:** Gather a sample from the population and compute relevant sample statistics.
- **Make the Decision:** Compare the calculated test statistic with the critical value from the corresponding statistical distribution. If the test statistic falls into the critical region, reject the null hypothesis; otherwise, accept it.

IV. SYSTEM MODEL, SECURITY ASSUMPTION AND DESIGN GOALS

A. System Model

We consider an NDN network consisting of three parts: content producers, routers, and users, as shown in Fig. 1.

- **Content producers (CPs):** They produce content, and generate a signature to guarantee its integrity. However, attackers can implement content poisoning attacks by pretending to be legitimate content producers and responding to interest packets with fake content.
- **Users:** They send interest packets specifying the content name to obtain desired content. Meanwhile, they verify each received data packet. If the verification fails, they will retransmit the same interest packet with the exclusion field containing the fake content's hash and signature, also called feedback packet. However, attackers can send malicious feedback.
- **Routers:** They forward interest packets and data packets. In addition, they can cache content passing through them, and directly respond to interest packets if there is matched

content in the caching. Moreover, they can verify data packets, check the feedback, send probing packets and analyze the returned content to find attackers. However, attackers can implement content poisoning attacks by compromising some routers and making them tamper with passing content, respond with fake content and even send malicious feedback like malicious users.

B. Security Assumption

The damage of content poisoning attacks studied in this paper, is quantified as r_{fc} , the ratio of fake contents received by users to the total contents received by users. In normal scenario, the authenticity of content is not influenced by attackers, so the number of fake contents received by users is 0, leading to r_{fc} of 0. In attack scenario, fake contents are generated and transmitted to users, making r_{fc} greater than 0. The larger r_{fc} is, the attack damage is more severe. Attackers can increase r_{fc} both directly and indirectly, that is, directly amplify damage by generating more fake content and indirectly sustain damage by weakening defenses. We model the attack behavior from the perspectives of attack method, attack object and attack initiator.

- **Attack method:** We consider common active attack methods that can amplify the damage or weaken the defense, including forgery, modification, replay, and Denial of Service (DoS) attacks.
- **Attack object:** Any type of NDN packet can be manipulated by versatile attackers, including data packets, normal interest packets and feedback packets. For data packets, forgery and modification can generate fake content, making users receive more fake content and less authentic content, thereby elevating r_{fc} . For normal interest packets, the attack methods have no impact on r_{fc} since interest packets cannot influence the authenticity of content. For feedback packets, which are special interest packets with information in the exclusion field, forgery, modification, replay, and the DoS attack can undermine the defense, preventing r_{fc} from decreasing.
- **Attack initiator:** The attack can be initiated by all entities in the network, including content producers (CPs), routers and users. Malicious CPs impersonate content producers and inject forged content, elevating r_{fc} . Compromised routers do not follow our scheme and they modify passing content and respond with forged content to increase r_{fc} , and may even send malicious feedback like malicious users to prevent r_{fc} from decreasing. Malicious users can send forged or modified feedback, or replay earlier sent feedback, or send excessive feedback to cause DoS, preventing r_{fc} from decreasing.

Table I shows the possible effective attack methods on different attack objects by different attack initiators to cause the damage of content poisoning attacks r_{fc} .

C. Design Goals

In this paper, we intend to design a secure and lightweight scheme against content poisoning attacks. Specifically, the proposed scheme achieves the following design goals:

TABLE I
POSSIBLE EFFECTIVE ATTACKS

Attack	Data Packet	Feedback Packet
Forgery	CP&Router	User&Router
Modification	Router	User&Router
Replay	-	User&Router
DoS Attack	-	User&Router

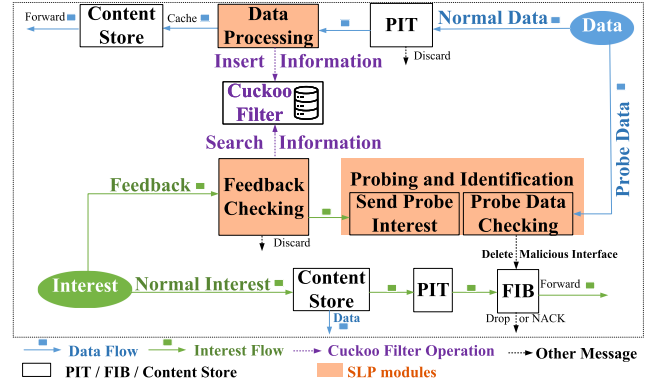


Fig. 2. System overview.

- **Resistance against content poisoning attacks.** To defend against content poisoning attacks, we need to quickly identify the attacker node and sever its connection with affected nodes. Also, fake content must be removed from contaminated caching to avoid its further spread. Moreover, users should only receive a minimal amount of fake content at the onset of the attack, ceasing thereafter.
- **Resistance against malicious feedback.** To prevent the harm from malicious feedback via forgery, modification, replay and DoS attacks, we need to identify such feedback at low costs.
- **High efficiency.** The proposed scheme is required to be efficient for routers to detect attacker nodes. Besides, users should be able to obtain desired content efficiently.

V. PROPOSED SCHEME

A. Overview

As shown in Fig. 2, SLP comprises two primary parts. The first part is the feedback processing module coordinated with the data processing module, and the second part is the probing and identification module.

The first part aims to obtain the probing timing and nodes from the authentic feedback. In the data processing module, crucial information about passing content is stored in a space-efficient cuckoo filter, the structure and utilization methods of which are modified to meet the requirement of design goals. In the feedback processing module, authentic feedback that contains received fake content's hash and signature, indicates the probing timing and nodes at low costs based on the cuckoo filter, while malicious feedback is identified and discarded.

The second part performs the probing process and identifies the attacker node. The router that has received the authentic feedback, sends specifically chosen interest packets through the designated interface so that the corresponding data packets'

authenticity corresponds one-to-one with the trustworthiness of a single node. Then these data packets undergo analysis via hypothesis testing, accommodating dynamic attackers and network instability. The analysis results indicates whether the next hop is the attacker node.

The data processing module is located between the PIT and the cache store, providing information to the feedback checking module via the cuckoo filter. The probing and identification module is after the feedback checking module, with its output influencing the FIB. These modules are independent of the PIT and content store since they do not modify the PIT entries or the cache. In contrast, FIB is changed by the probing and identification module. Specifically, if the probing and identification module informs the router that one of its neighbor routers is malicious, it will delete the interface connected to the malicious router from the FIB.

B. Data Processing in Routers

The data processing (DP) module serves as a preparation for the feedback processing (FP) module. FP has two objectives: checking the feedback's authenticity; informing the router of the interface from which it previously received the fake content in the feedback. So DP aids FP in fulfilling the two objectives.

1) The preliminary algorithm to achieve the two objectives.

For the first objective, we need to decide whether to verify the feedback itself or verify the content in the feedback. For the former way, if the feedback is verified as authentic, we can accept it, since normally attackers cannot forge a legitimate signature. However, users are not reliable, who may leak the signing keys for economic reasons, resulting in attackers being able to generate legitimate signatures. Conversely, for the latter way, if the content in the feedback is verified as fake, accepting the feedback appears reasonable, since the feedback serves to reflect the fake content appearing in the network previously. However, invalid signatures are easy to generate, so attackers can construct numerous feedback, all of which will be accepted. The latter way seems to fail, but we observe that as long as the fake content in the feedback is confirmed to have been received previously, it is correct to accept the feedback. Therefore, we need to store content when receiving each data packet and search for it when receiving each feedback. However, the large quantity of content and the substantial size of each content make traditional storage structures impractical. As a solution, we utilize the cuckoo filter, an efficient structure for membership test, to store the content received historically.

For the second objective, we modify the structure of the cuckoo filter to achieve it. Obtaining the content's previous arrival interface by looking up the content is actually a $\langle \text{key}, \text{value} \rangle$ mapping process. However, the original cuckoo filter can only inform the presence of an item, but cannot perform the $\langle \text{key}, \text{value} \rangle$ mapping. To break the limit, we explore the searching logic of the cuckoo filter, that is, if an item's fingerprint is found in specific buckets, it confirms the item's presence. Hence, if we store the complete interface value along with the fingerprint, we can easily know the exact interface when finding the item's fingerprint. Note that we choose cuckoo filters rather than other Approximate Membership

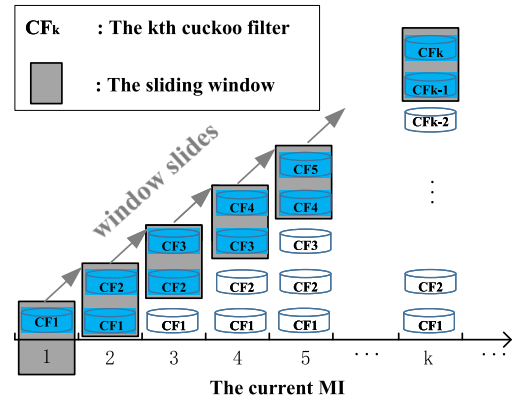


Fig. 3. Updating cuckoo filters based on the sliding window.

Query (AMQ) filters for two reasons. First, simple filters like Bloom filters and Counting Bloom filters cannot be extended to store the complete interface value. Second, among filters that meet the above requirements, cuckoo filters perform best in terms of space costs and false positive rate as per [32].

2) The optimized algorithm to achieve the two objectives.

Although the two objectives are achieved, opportunities exist to further reduce storage costs. We observe that content does not need to be retained in the cuckoo filter beyond the duration Δt between content reception and the subsequent corresponding feedback reception. However, simple solutions cannot achieve this efficiently. For example, establishing individual timers for each item and deleting them upon timer expiration introduces excessive computational overhead. Another infeasible solution is periodic removal of the cuckoo filter, which makes most feedback especially those arriving shortly after the removal, lack a qualified cuckoo filter storing content received over the past Δt . We recognize that one cuckoo filter is not enough if it may be removed at some moment. Therefore, we design a sliding-window-based stream algorithm to update them.

As Fig. 3 shows, time is divided into consecutive monitor intervals (MIs) with a duration of Δt . At the start of each MI, a new empty cuckoo filter is constructed to store content received during that MI. Consequently, each cuckoo filter corresponds to each MI. Meanwhile, a sliding window is used to retain the necessary cuckoo filters, whose size is two MIs, or equivalently, two cuckoo filters. We consider only two MIs because each cuckoo filter needs to be retained for at least two MIs to ensure that all items in the cuckoo filter, especially the latest one, remain for at least one MI. At the start of each MI, the window shifts forward by one MI to contain the latest two cuckoo filters, and cuckoo filters outside the window are deleted as a whole. Note that the deletion operation is performed at the level of cuckoo filters, not at the level of items. Therefore, the sliding window holds cuckoo filters storing content received within at least an MI in the past, enabling searching whenever feedback arrives. Moreover, the storage costs are limited to the size of two cuckoo filters, each only storing the content arriving within an MI.

In summary, we firstly apply the cuckoo filter to check feedback efficiently and effectively, then redesign the cuckoo filter to facilitate searching the content's previously incoming

interface, and lastly update cuckoo filters based on the sliding window to dramatically reduce storage costs. As **Algorithm 1** shows, upon reaching the commencement of the k th MI, construct an empty cuckoo filter cf_k . If there are three cuckoo filters, remove the oldest cuckoo filter cf_{k-2} to make the sliding window hold at most two cuckoo filters all the time. For every arriving data packet, find an available entry for the concatenation of content's hash and signature $h||sig$ in the newest cuckoo filter cf_k , and insert the concatenation of its fingerprint and the arriving face $f(h||sig)||face$ to that entry.

Algorithm 1 Data Processing Algorithm

Input: The arriving content c 's hash h , signature sig and incoming interface $face$, the current time now , and the fingerprint generation function f .

Output: The newest set of cuckoo filters.

```

1 if  $now$  is the start moment of the  $k$ th MI then
2   Construct an empty cuckoo filter  $cf_k$ ;
3   if  $k \geq 3$  then
4     Remove  $cf_{k-2}$ ;
5   end
6 end
7 for each incoming  $c$  do
8   if  $now$  is in the  $k$ th MI then
9     Find an available entry  $e$  for  $h||sig$  in  $cf_k$ ;
10    Insert  $f(h||sig)||face$  to  $e$ ;
11  end
12 end

```

C. Feedback Processing in Routers

Besides clearing contaminated caches, the feedback processing (FP) module serves as a trigger and guide for the probing and identification (PI) module. Specifically, for one thing, FP notifies the router that the content poisoning attack has happened and triggers the initiation of PI. For another, FP informs the router of the interface that previously received the fake content and guides it to probe the neighbor node connected to that interface. However, this relies on the feedback's authenticity. Typically, users generate the authentic feedback with received fake content and its signature, but malicious users and compromised routers can generate malicious feedback in various ways to harm our system.

We categorize the ways malicious feedback can be generated into four types and outline how to identify each type. The first way is forgery. Attackers can produce content and its signature that were never received, which are used to forge the feedback, misleading routers into believing the attack has happened. It can be solved by searching in cuckoo filters (CFs). Also, attackers can forge the feedback with received authentic content and its signature, misleading routers into believing the content is fake and the attack has happened. It can be solved by signature verification. The second way is modification. Attackers modify either a received content object or signature to generate the feedback, misleading routers into believing the attack has happened. It can be solved by searching in CFs. The third way is replay. Attackers replay

previously authentic feedback, misleading routers to believe the attack happens again. It can be solved by deleting in CFs. The last way is Denial of Service (DoS) attacks. Attackers can generate and send a large volume of malicious feedback by use of the aforementioned ways, or collect many authentic feedback packets and send them within a short period, making routers exhausted by the check of feedback. It can be solved by searching in the CFs. The details of the resistance against the above attacks are presented in Section VI.

What's more, we design a simple token-based method to further decrease feedback checking costs. Every router maintains a token bucket for each of its neighbor nodes, initially containing one token. Increase n_{inc} tokens every time the neighbor node sends an authentic feedback and decrease n_{dec} tokens every time the neighbor node sends a malicious feedback, with n_{dec} is greater than n_{inc} . If a neighbor node's token bucket is empty, directly discard the feedback from it. In this way, if the attacker node sends too many feedback packets, no check on the feedback will be performed.

Algorithm 2 shows how to process the feedback packet. Discard the feedback if the token bucket of the neighbor node that sends the feedback is empty. Otherwise, extract content's hash h and signature sig in the feedback, concatenate them and compute the fingerprint $f(h||sig)$. Search the entry e in the CFs whose value after removing the last b_{face} bits is $f(h||sig)$. If e is found and then sig is verified as invalid, it proves that the feedback is authentic, that is, the content is fake and was received previously. So we extract the last b_{face} bits of e as the interface receiving the content previously, delete e from CFs to avoid the replay attack, remove this fake content from the caching, and store the feedback for later forwarding. If the check fails, the feedback is malicious.

D. Probing and Identification in Routers

The goal of the probing and identification (PI) module is to precisely identify which node is the attacker. Existing schemes try to achieve this by analyzing the authenticity of data packets received from each neighbor node. However, this is misleading for three reasons. Firstly, an honest node is possible to send fake content ("Honest Node Sends Fake Content" for short) when it forwards fake content without verification or responds with contaminated caching. Secondly, even though it sends authentic content, the content could turn to be fake ("Authentic Content Turns Fake" for short) when received by the next hop due to transmission errors. Lastly, an attacker node is possible to send authentic content ("Attacker Sends Authentic Content" for short) if it attacks probabilistically. We need to deal with the above three misleading circumstances to accurately distinguish between honest nodes and attacker nodes.

1) Deal With "Honest Node Sends Fake Content".

An honest node could forward fake content due to a lack of verification. Our goal is to ensure that received content's authenticity is not influenced by it. So we only focus on the content that is not forwarded without verification. Considering the untrustworthiness of other nodes, a router cannot trust the previous hop's claim of having verified the content. Therefore, we straightly focus on the content that is not forwarded but responded by the previous hop.

Algorithm 2 Feedback Processing Algorithm

Input: Content c 's hash h and signature sig in the feedback fb , the set of cuckoo filters $\{CF\}$ in the current window, the interface's length b_{face} , the fingerprint generation function f , the token bucket tb of the neighbor node sending fb .

Output: {Whether fb is authentic, the interface that received c previously}

```

1 if  $tb$  is empty then
2   | Discard the feedback;
3 else
4   | if find such entry  $e$  in  $\{CF\}$ , that
      |  $(e \gg b_{face}) == f(h||sig)$  then
5     | if  $sig$  is verified as invalid then
6       |  $face = e \ \& \ (2^{b_{face}} - 1)$ ;
7       | Delete  $e$  from  $\{CF\}$ ;
8       | Remove  $c$  in the caching;
9       | Increase  $n_{inc}$  tokens in  $tb$ ;
10      | Store  $fb$ ;
11      | return {authentic,  $face$ };
12    | end
13  | end
14  | Decrease  $n_{dec}$  tokens in  $tb$ ;
15  | return {malicious, null};
16 end

```

Now we need to judge which content is responded by the previous hop. Similarly, a router cannot trust the previous hop's claim that the content is responded by it. Instead, it needs to make judgments relying on its own knowledge. When exploring the transmission logic in NDN, we observe that it is because the router sends an interest packet to its upstream neighbor router that it receives the corresponding data packet. Typically, routers only forward interest packets from users, lacking control over whether the interest packet can be responded by the upstream neighbor router. However, routers are able to generate interest packets by their own and send them to neighbor nodes. In particular, if a router proactively sends an interest packet to a neighbor node for content already cached, the neighbor node will respond to it directly. This process, called probing, allows the router to distinguish between responded content and forwarded content from the upstream neighbor router. To facilitate the probing process, the router requires knowledge of the content present in the neighbor node's caching. This has been studied in other work and is achievable in most caching strategies [34], [35]. For simplicity, we illustrate the method to ascertain the neighbor node's caching under the defaulted caching strategy that routers cache all passing content. The router maintains a queue capable of holding n_{probe} content names, storing content names in it with a probability of $p = \frac{n_{probe}}{cacheSize}$ after receiving content from the neighbor node, where n_{probe} denotes the number of probing packets required, and $cacheSize$ refers to the caching size. Upon reaching full capacity in the queue, the oldest item is removed. This ensures the queue always

maintains enough content names matching the neighbor node's cached content regardless of caching replacement strategies.

Next we deal with the scenario where an honest node responds with contaminated caching. During an attack, fake content is inevitably stored in the caching without verification. Given that ensuring the authenticity of content when *stored into* the caching isn't feasible, our focus shifts to guaranteeing the authenticity of content when *retrieved from* the caching. In other words, when an interest packet hits the content in the caching, the content should be verified before responding to the interest packet. The verification costs of the content hit in the caching are acceptable as it constitutes less than 10% of the cached content [13]. What's more, we design a verification scheme as follows to eliminate duplicate verifications, which further reduces the computation costs.

Generally, popular content is requested significantly more times than the less popular content in NDN. Consequently, the content in the caching is predominantly popular content, which is hit and verified many times. To alleviate the burden of frequent repetitive verification of popular content in the caching, we tag the content as authentic after verifying it for the first time. However, owing to the limited capacity of the caching, popular content is probable to be evicted and reinserted. In such a case, the reinserted popular content still has to be verified when it is hit later. To tackle this, we exploit the cuckoo filter again to store content that is hit in the caching and verified as authentic. When the content is hit, first check if it has the tag representing that it has been verified as authentic, and then check if the concatenation of its hash and signature $h||sig$ is in a cuckoo filter for verification cf_{veri} , and lastly check if sig is valid. Passing any of these checks implies content authenticity, thus stopping subsequent checks. The router tags it if untagged and adds $h||sig$ to the cuckoo filter if absent. Conversely, failure of all these checks indicates that the content is fake, prompting the router to remove it from the caching.

2) Deal With "Authentic Content Turns Fake" and "Attacker Sends Authentic Content".

Authentic content could turn to be fake due to transmission errors [36], and attackers could send authentic content if they execute the attack probabilistically. We cannot control the transmission error rate and attackers' behaviors, so our focus is on eliminating their influence on the probing results. In these circumstances, the authenticity of an individual data packet fails to conclusively reflect the trustworthiness of the sending node. Consequently, statistical results become imperative to determine whether the sending node is an attacker.

Hypothesis testing, a statistical technique used to decide the validity of the null hypothesis among two predefined hypotheses, has a notable advantage in providing a preset significance level α to quantify the decision's reliability. However, α solely measures the probability of rejecting a true null hypothesis, termed Type I error, while the probability of accepting a false null hypothesis, termed Type II error, remains uncertain. Here, we combine a pair of interrelated hypothesis testing to quantify Type II error, with each of the two null hypotheses being opposite. Suppose a router sends n probing packets. X denotes whether the content is fake, which is 1 for fake content,

and 0 otherwise. Denote s as the sum of X values of all content. Assume that whether a content object is fake follows a Bernoulli distribution $B(1, p)$. The hypothesis testings for the two circumstances are as follows.

For the circumstance of “Authentic Content Turns Fake”, we formulate two hypotheses $\{H\}$: $H_0 : p \leq p_0$; $H_1 : p > p_0$. The null hypothesis H_0 represents that the node is honest, while the alternative hypothesis H_1 represents that the node is an attacker. p_0 is the maximum transmission error rate. We utilize the rate hypothesis testing to make the decision Dec :

$$Dec = \begin{cases} \text{reject } H_0, & \text{if } p_0 \leq p_\alpha(s) \\ \text{accept } H_0, & \text{if } p_0 > p_\alpha(s). \end{cases} \quad (1)$$

Here $p_\alpha(s)$ is computed from s :

$$p_\alpha(s) = \frac{1}{1 + \frac{n-s+1}{s} F_{1-\alpha}(2(n-s+1), 2s)}, \quad (2)$$

where $F_\gamma(n_1, n_2)$ is the γ quantile of the F distribution with degrees of freedom n_1 and n_2 .

For the circumstance of “Attacker Sends Authentic Content”, we formulate two hypotheses $\{H'\}$: $H'_0 : p \geq p'_0$; $H'_1 : p < p'_0$. The null hypothesis H'_0 represents that the node is an attacker, while the alternative hypothesis H'_1 represents that the node is honest. p'_0 is the lowest attack probability of an attacker. We utilize the rate hypothesis testing to make the decision Dec' :

$$Dec' = \begin{cases} \text{reject } H'_0, & \text{if } p'_0 \leq p_\alpha(s)' \\ \text{accept } H'_0, & \text{if } p'_0 > p_\alpha(s)'. \end{cases} \quad (3)$$

Here $p_\alpha(s)'$ is computed from s :

$$p_\alpha(s)' = \frac{1}{1 + \frac{n-s}{s+1} \frac{1}{F_{1-\alpha}(2(s+1), 2(n-s))}}, \quad (4)$$

where $F_\gamma(n_1, n_2)$ is the same as in formula (2).

For each s , compute $p_\alpha(s)$ and $p_\alpha(s)'$ to make decisions Dec and Dec' . However, the two decisions may conflict, that is, one hypothesis testing might conclude that the node is honest while another might deem it malicious. This stems from the uncertainty of Type II error. Augmenting the sample size n_{probe} can control the magnitude of Type II error, but increase the probing costs. Therefore we design a simple offline algorithm to determine the minimum n_{probe} that ensures both types of errors are below the aimed value. Specifically, gradually increase n_{probe} until both hypothesis testings reach concordant conclusions for any value of s , that is, the conclusion of rejecting H_0 matches the conclusion of accepting H'_0 and vice versa. At this point, the Type II error of any hypothesis testing is limited by the known Type I error α of its interrelated hypothesis testing. Since the algorithm traverses from 1 to choose the first n that makes the two hypothesis testings reach concordant decisions for any value from 1 to n , the complexity is $O(n_{probe}^2)$, where n_{probe} is the result of the algorithm. The computation overhead is tolerable because the algorithm is offline and n_{probe} is not large.

Thus far, we have dealt with all the three misleading circumstances: “Honest Node Sends Fake Content”, “Authentic

Algorithm 3 Probing and Identification Algorithm

Input: Interface $face$, the queue Q , content recently received c_{last} , the number of probing packets n_{probe} , the corresponding received content c , the stored feedback $\{fb\}$ matching $face$.

Output: Honest or Malicious.

```
// Derive the value of  $n_{probe}$ 
1 for ( $n = 1; n \leq INTMAX; n++$ ) do
2   for ( $k = 1; k \leq n; k++$ ) do
3     Compute  $p_\alpha(k)$  and  $p_\alpha(k)'$  as in (2), (4);
4     Make decisions  $Dec$  and  $Dec'$  as in (1), (3);
5     if  $Dec$  doesn't match  $Dec'$  then
6       break;
7     end
8   end
9   if  $k > n$  then
10    go to line 13;
11  end
12 end
13  $n_{probe} = n$ ;
   // Probe and identify
14 Generate  $n_{probe}$  interest packets  $\{Probe_{pac}\}$  from  $Q$ ;
15 Transmit  $\{Probe_{pac}\}$  through  $face$ ;
16 Initialize  $s$  and  $num_{rx}$  as 0;
17 for each incoming  $c$  do
18    $num_{rx} += 1$ ;
19   if ( $c$  is fake) then
20      $s = s + 1$ ;
21   end
22   if  $num_{rx} == n_{probe}$  then
23     if  $p_0 > p_\alpha(s)$  then
24       Transmit  $\{fb\}$  through  $face$ ;
25       return Honest;
26     else
27       Exhibit receiving packets from  $face$ ;
28       return Malicious;
29     end
30   end
31 end
```

Content Turns Fake” and “Attacker Sends Authentic Content”.

Algorithm 3 shows how to probe and identify attackers accurately. From lines 1 to 13, derive the minimum value of n_{probe} that makes Type II error below α by ensuring the two hypothesis testings output consistent conclusions. Then extract the probing interface $face$ from the feedback processing algorithm. Generate n_{probe} interest packets from the queue Q that stores cached content's names of the neighbor node connected to $face$, and transmit them through $face$ for probing. When receiving n_{probe} corresponding data packets, check their authenticity. Then compare p_0 and $p_\alpha(s)$ to determine whether the neighbor node is honest. If it is honest, transmit stored feedback through $face$. Otherwise, prohibit packet transmission with it. Note that the feedback is forwarded only after the probing process is completed, which prevents the attacker's awareness of the probing.

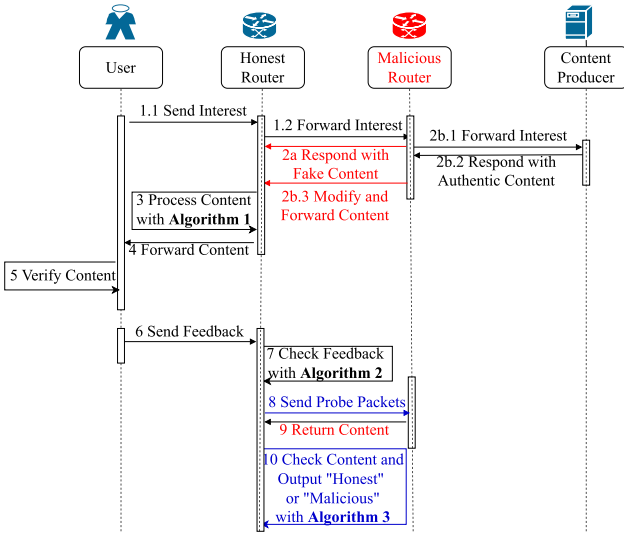


Fig. 4. Time diagram.

In summary, we utilize users' feedback to determine the appropriate probing timing and nodes, and then perform the probing process and identify the attacker nodes. Fig. 4 shows the time diagram of the protocol, where the number of nodes on the path is simplified for clarity. When the malicious router receives a user's interest packets, it directly responds with fake content, or modifies the received content and forwards it. The honest router processes the content with **Algorithm 1**. The user verifies the content after receiving it, and sends the feedback packet if the content is fake. The honest router checks the feedback with **Algorithm 2** after receiving it. If the feedback passes the check, the honest router sends probe packets and checks the returned content to decide whether the upstream neighbor router is honest or malicious using **Algorithm 3**.

E. Discussion

If malicious routers are intelligent enough, they will try to behave normally for the probe packets, but behave maliciously for the non-probe packets. However, our proposed scheme shows resistance against them from the perspective of the attacker's cost-benefit ratio. Specifically, probe packets and non-probe packets have the same form and payload, and the probe packets are randomly selected by a victim router from the content recently received from the suspicious router. Therefore, only this victim router knows which packets are finally chosen as the probe packets. If the attacker wants to behave normally for the probe packets, it must also behave normally for all interest packets corresponding to content that it recently sent. Since there is a considerable temporal locality in users' requests [37], attackers can only cause insignificant damage. Additionally, compromising routers is difficult for attackers [13], requiring significant effort. When attackers can only inflict minimal damage, they are unlikely to launch content poisoning attacks, considering the substantial costs involved in compromising routers.

Furthermore, we discuss a supplement to the original scheme to completely address the evasion of such intelli-

gent attackers from detection. Specifically, we introduce an additional check named consistency check after the original check. When the suspicious router receives the probe packet and sends the corresponding content c_{now} to the victim router, the victim router first checks its authenticity and then checks if it is consistent with the last received content c_{last} of the same name. With the newly proposed dual check and the originally used hypothesis testing, if attackers want to escape detection during the probe process, they have to ensure c_{now} and c_{last} are both authentic, which means they have to behave normally to all packets. Therefore, attackers cannot evade detection if they inject fake content.

Still, it is reasonable to propose the defense mechanism in the original adversary model in this paper, since this intelligent malicious router is rarely considered in related works due to fixed attack modes and program limitations in realistic environments. Specifically, in most literature involving malicious routers, attackers are assumed to perform the same attack behaviors for each packet, e.g., [38] and [39].

VI. SECURITY ANALYSIS

A. Resistance Against Malicious Feedback

A router identifies a feedback packet received at time t_1 as authentic when meeting three conditions:

- The router has previously received the content and signature in the feedback at a certain time t_0 ;
- The difference between t_1 and t_0 is within $2*MI$;
- The signature is not a valid signature of the content.

Lemma 1 (Sufficient Accuracy of Cuckoo Filters): *In the feedback processing module, a router uses a set of modified cuckoo filters recording passing content cf_{pass} , each storing the fingerprint of its hash and signature's concatenation ($h||sig$) and the arrival interface $face$ during a MI. The accuracy of cf_{pass} is sufficient to guarantee that the lookup results are correct with a negligible error probability.*
Proof: In cf_{pass} , assume each bucket has b entries, and the fingerprint' length is l bits. The false positive rate fp satisfies:

$$fp \leq 1 - \left(1 - \frac{1}{2^l}\right)^{2b}, \quad (5)$$

□

which can be made arbitrarily small by adjusting the values of b and l . The basic paper of the cuckoo filter [32] concludes that when the target false positive rate fp decreases to $0.00001 < fp \leq 0.002$, four entries per bucket minimizes space. So b is set as 4. Since $fp \leq 1 - \left(1 - \frac{1}{2^l}\right)^{2b}$, we set the fingerprint's length l as 13 to make $fp < 0.001$. Therefore, if the content's $h||sig$ is found in cf_{pass} , it is ensured that the content has passed through the router previously with the error probability less than 0.001. Besides, due to the property of the cuckoo filter, if the content's $h||sig$ is not found in cf_{pass} , it is guaranteed with 100% certainty that the content has not passed through the router previously.

Next we analyze the resistance against the possible attacks related to malicious feedback.

Forgery Attack Resistance: An adversary $\mathcal{A}$ attempts to forge the feedback in two ways. In the first way, $\mathcal{A}$ generates previously unreceived content and its signature, and uses them to forge the feedback. However, when the router looks up the concatenation of the content's hash and signature ($h||sig$) in cf_{pass} , the router does not find it and thus will reject the feedback based on Lemma 1. In the second way, $\mathcal{A}$ uses received authentic content and signature to forge the feedback. However, when the router verifies the signature, it recognizes the content as authentic and thus will reject the feedback.

Modification Attack Resistance: An adversary $\mathcal{A}$ attempts to modify the signature of a received content object, and uses the original content and modified signature to generate the feedback. Or $\mathcal{A}$ attempts to modify a received content object, and uses the modified content and its original signature to generate the feedback. However, in any way, when the router looks up the concatenation of the content's hash and signature ($h||sig$) in cf_{pass} , the router does not find it and thus will reject the feedback based on Lemma 1.

Replay Attack Resistance: An adversary $\mathcal{A}$ attempts to reserve a copy of an authentic feedback packet and sends the feedback again. However, when the router receives the authentic feedback for the first time, it will delete the concatenation of the content's hash and signature ($h||sig$) in cf_{pass} . So when the router receives the feedback again and looks up ($h||sig$) in cf_{pass} , the router does not find it and thus rejects the feedback based on Lemma 1.

Denial of Service (DoS) Attack Resistance: An adversary $\mathcal{A}$ attempts to generate and send a large volume of malicious feedback through the aforementioned forgery, modification and replay attacks, or delay the sending of many authentic feedback packets and transmit them together within a short period, making routers exhausted by the check of feedback. However, the majority of malicious feedback and delayed feedback can be identified efficiently by looking up ($h||sig$) in the cf_{pass} that only stores fresh content, the costs of which are in 10 nanoseconds level. The only considerable costs lie in verifying the signature in those feedback generated by received authentic content and signature. However, the number of received authentic contents is limited during a period of time, so the costs are limited. Moreover, the token-based method guarantees that the number of malicious feedbacks needed to check is $\frac{n_{inc}}{n_{dec}}$ times smaller than the number of authentic feedbacks, while the number of authentic feedbacks is minimal since we block attackers very quickly.

Therefore, we can resist malicious feedback.

B. Resistance Against Content Poisoning Attacks

After the feedback processing module indicates a router whose neighbor node has sent fake content, the router starts probing that neighbor node.

Lemma 2 (Sufficient Accuracy of the Hypothesis Testing): The hypothesis testing used in our scheme outputs correct results with a negligible error probability.

Proof: The errors in the hypothesis testing are divided into two types. Type I error happens when rejecting the correct null hypothesis, while Type II error happens when accepting the false null hypothesis. The predefined α limits the Type

I error probability. We use a pair of interrelated hypothesis testing $\{H\}$ and $\{H'\}$ to control the Type II error probability.

In $\{H\}$, the null hypothesis H_0 signifies the node is honest and the alternative hypothesis H_1 means the node is an attacker. Conversely, in $\{H'\}$, the null hypothesis H'_0 indicates the node is an attacker and the alternative hypothesis H'_1 implies the node is honest. Therefore, the Type I error in $\{H\}$ corresponds to the Type II error in $\{H'\}$ and vice versa. We augment the sample size n_{probe} to reduce the Type II error probability until the two hypothesis testing output the same result. At this point, the Type II error probability in $\{H\}$ is limited by the Type I error probability α in $\{H'\}$, and conversely the Type II error probability in $\{H'\}$ is constrained by the Type I error probability α in $\{H\}$. Therefore, Type II error probabilities of $\{H\}$ and $\{H'\}$ are all less than α .

The qualitative analysis outlined above can be complemented by probability theory techniques to quantitatively compute the Type II error probability. For the first hypothesis testing, the Type II error probability $ep2$ is:

$$ep2 = \sum_{k \in K} p(s = k), \quad (6)$$

where $K = \{k | p_\alpha(k) \leq p_0\}$, $p(s = k) = \binom{n_{probe}}{k} p_A^k (1 - p_A)^{n_{probe}-k}$, and $p_\alpha(k)$ is computed as formula (2). p_A is an attacker's probability of sending fake content that is assumed to be at least 0.5. p_0 is the highest transmission error probability that is assumed to be 0.01. Similarly for the second hypothesis testing, the Type II error probability $ep2'$ is:

$$ep2' = \sum_{k \in K'} p(s = k), \quad (7)$$

where $K' = \{k | p_\alpha(k)' \geq p'_0\}$, $p(s = k) = \binom{n_{probe}}{k} p_T^k (1 - p_T)^{n_{probe}-k}$, and $p_\alpha(k)'$ is computed as formula (4). p_T is the transmission error probability that is assumed to be at most 0.01. p'_0 is the lowest probability of an attacker sending fake content that is assumed to be 0.5. With α and n_{probe} set as 0.01 and 11 respectively, $ep2$ and $ep2'$ are all less than 0.01.

The qualitative and quantitative analysis indicate that the Type II error probability is less than 0.01. Since the Type I error probability is less than α , which is 0.01, we state that the hypothesis testing can output accurate results with the error probability less than 0.01. $\square$

Next we analyze the resistance against the possible circumstances related to content poisoning attacks.

Probing Unaffected By "Honest Node Sends Fake Content": Although an honest node might send fake content, the probing process is not affected. When an honest node forwards content without verification or responds with contaminated caching, it might send fake content. However, the probing packets are generated based on the caching of the neighbor node, so they will definitely be responded by the neighbor node. Besides, an honest node will verify the content hit in the caching. Therefore, if the returned content responding to probing packets is fake, it is not because the honest neighbor node sends it.

Probing Unaffected By "Authentic Content Turns Fake" and "Attacker Sends Authentic Content": Although the authentic content might turn fake and an attacker might

send authentic content, the probing process is not affected. Authentic content might turn to be fake due to transmission errors, and an attacker might send authentic content if it executes the attack probabilistically. However, we use a pair of interrelated hypothesis testing to exclude their impact on the probing results. According to Lemma 2, even though the circumstances of “Authentic Content Turns Fake” and “Attacker Sends Authentic Content” exist, the probing result is not affected.

Resistance against Collusion Attacks: There is at least one probing path between a legitimate user and a malicious node $\mathcal{A}$ that injects fake content. In the collusive attack scenario, the collusive node can be located on the path, called $\mathcal{A}_{in}$, or out of the path, called $\mathcal{A}_{out}$. They can manipulate normal interest packets, data packets, feedback packets and probe packets. Among them, feedback packets and probe packets are key in the probing process, while manipulating normal interest packets and data packets is useless. First we analyze $\mathcal{A}_{in}$. Blocking feedback packets from transmitting upstream can delay the detection of $\mathcal{A}$. However, $\mathcal{A}_{in}$ sacrifices itself and is detected. In addition, though $\mathcal{A}$ is not detected on this path, it is probably detected on other paths where $\mathcal{A}$'s fake content is transmitted. Even if there exists $\mathcal{A}_{in}$ on all paths to protect $\mathcal{A}$, $\mathcal{A}$ will be detected in the next probing process when it is the closest malicious router to users since $\mathcal{A}_{in}$ was disconnected. Besides manipulating feedback packets, $\mathcal{A}_{in}$ can cancel sending probing packets to protect $\mathcal{A}$ if $\mathcal{A}$ is the upstream neighbor of $\mathcal{A}_{in}$. However, $\mathcal{A}_{in}$ is detected since there is no downstream collusive router to protect it, and thus $\mathcal{A}$ will be detected in the next probing process. Overall, $\mathcal{A}_{in}$ can only delay the detection of $\mathcal{A}$ but cannot prevent it. Next we analyze $\mathcal{A}_{out}$. $\mathcal{A}_{out}$ can do nothing to protect $\mathcal{A}$. For one thing, $\mathcal{A}_{out}$ is out of the path and thus cannot affect the transmission of feedback packets and probe packets on the path. For another, it is useless for it to send forged feedback packets or probe packets to the router on the path, since forged feedback packets will be checked and discarded, and forged probe packets have no impact on this path. Therefore, collusion cannot enable attackers to evade detection. The underlying rationale is that, unlike schemes that rely on multiple nodes' coordination to make decisions, our scheme detects attackers by each router's individual probing results, thus inherently immune to collusion attacks.

Therefore, we can resist the content poisoning attack.

VII. PERFORMANCE ANALYSIS

In this section, we analyze the overhead and test the performance of multiple metrics in various scenarios.

We implement SLP using ndnSIM 2.9 [40]. The topologies are generated using BRITE, where the number of users is 10% of routers. The link has a 1 Gbps bandwidth and a 5 ms delay. We use the best-route forwarding protocol, and the LRU cache replacement strategy with the cache size of 1000. The experiments are conducted on Ubuntu 20.04 with a 2.7 GHz Intel Core i5 processor and 16 GB RAM. The comparative schemes are ELC [29] and Kim's scheme [13] for two reasons. Firstly, ELC and Kim's scheme are strongly related to our proposed scheme SLP, whereas some other schemes

TABLE II
PARAMETER SETTING

Parameter	Symbol	Value
monitor interval	MI	1 s
fingerprint's length	l	13 bits
interface's length	b_{face}	3 bits
number of entries per bucket	b	4
number of increased tokens	n_{inc}	1
number of decreased tokens	n_{dec}	5
significance level in hypothesis testing	α	0.01

are only cryptographic signature algorithms or provide an abstract model without practical algorithms. Secondly, ELC and Kim's scheme are relatively more effective and secure in the scenario with compromised routers, showing lower attack damage and higher user service quality. In contrast, some schemes still verify each data packet, causing unaffordable overhead, and some schemes cannot work at all under the scenario with malicious routers. The implementation of cuckoo filters in this paper refers to [32], where the hash function used is “TwoIndependentMultiplyShift”, which takes a 64-bit number as input. Parameters are set as shown in Table II. The fingerprint length is set as 13 bits, and we explain the reason in the following storage overhead analysis. The interface's length is the length of the bit string used to encode interface numbers.

A. Overhead Analysis

1) *Computation Overhead:* Since the verification overhead is the main part in the comparative schemes, we segment the overall computation overhead into verification overhead t_{verify} and other computation overhead t_{other} . Please note that the following numerical results are measured from the individual experiment rather than the simulations.

In SLP, the verification overhead occurs when a content is hit in the caching but has not been verified previously. Thus t_{verify} is expressed as $t_1 * n_{hit} * p_{nv}$, where t_1 represents the verification time for a single signature, n_{hit} is the number of cache hits, p_{nv} is the probability that a hit cached content has never been verified before. In addition, a router needs to insert each content's hash, signature, and incoming interface into a cuckoo filter, and probabilistically insert each content's name into a queue. The computation time for these operations is denoted as t_2 , which is less than 0.1 μ s. Consequently t_{other} is given by $t_2 * n_c * n_r$, where n_c is the number of contents, n_r is the average number of routers on the path.

In Kim's scheme, when hit in the unprotected region, the content is verified and inserted into the protected region, and another content in the protected region is moved to the unprotected region. t_{verify} is $t_1 * n_{hit} * p_{unp}$, where t_1 is the verification time for a signature, n_{hit} is the number of cache hits, p_{unp} is the probability that a hit cached content is in the unprotected region. t_{other} is the extra caching overhead $t_{cache} * n_{hit} * p_{unp}$, where t_{cache} is a content's caching time.

ELC adopts Kim's scheme, so it incurs the same overhead. Moreover, edge routers and the routers connected to them need to verify each passing content. Thus t_{verify} is $t_1 * (n_{hit} * p_{unp} + n_{cfcp} * 2)$, where n_{cfcp} is the number of contents from content producers. Besides, core routers periodically exchange

TABLE III
COMPUTATION OVERHEAD

Scheme	Verification Overhead	Other Overhead
SLP	$t_1 * n_{hit} * p_{nv}$	$t_2 * n_c * n_r$
Kim's	$t_1 * n_{hit} * p_{unp}$	$t_{cache} * n_{hit} * p_{unp}$
ELC	$t_1 * (n_{hit} * p_{unp} + n_{cfcp} * 2)$	$t_3 * n_{prd}$

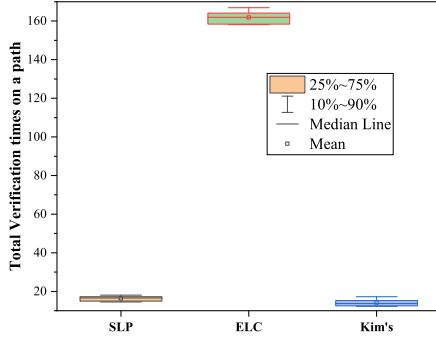


Fig. 5. Total verification overhead on a path.

information and reach consensus using blockchain, the computation time of which is denoted as t_3 . Thus t_{other} is $t_3 * n_{prd}$, where n_{prd} is the number of periods. Therefore, the overall computation overhead is $t_1 * (n_{hit} * p_{unp} + n_{cfcp} * 2) + t_3 * n_{prd}$.

As presented in Table III, in terms of verification overhead, SLP and Kim's scheme are close, while ELC causes much greater overhead. To intuitively demonstrate the differences, we test the total verification overhead for all routers on each path over 1 second, as shown in Fig. 5. ELC's verification overhead on a path is significantly greater than that of SLP and Kim's scheme, since it requires edge routers and edge-connected routers to verify each content. In terms of non-verification overhead, since the value of t_2 is far bigger than t_3 and t_{cache} , ELC's non-verification overhead is obviously greater than SLP and Kim's scheme.

The above is the overhead when no attack happens. The computation overhead of Kim's scheme and ELC do not change if the attack occurs. However, for SLP, if the content poisoning attack happens, a router needs to process the feedback and perform probing. Since the attack can be detected quickly, the number of feedback packets and probe packets is small, making the overhead negligible.

2) *Storage Overhead*: Our scheme uses a queue to store n_{probe} content names, two cuckoo filters for passing content cf_{pass} and a cuckoo filter for verification cf_{veri} . The queue size is 17B. According to [32], setting the number of entries per bucket, b , as 4 minimizes space when the aimed false positive rate fp is less than 0.002. Since $fp \leq 1 - (1 - \frac{1}{2^b})^{2b}$, we set the fingerprint's length l as 13 to make $fp < 0.001$. For cf_{pass} , each entry costs $l + b_{face} = 16$ bits, where b_{face} is the length of inserted interface. Each cf_{pass} stores passing data within 1 s, so for a router that forwards all data packets in the network, the storage costs of two cf_{pass} are 11.72 KB, while for most routers the storage costs are at the 1 KB level. For cf_{veri} , each entry costs $l = 13$ bits. The number of entries stored in cf_{veri} is equal to the cache size, so the storage costs are 1.59 KB. Therefore, the overall storage size of most routers is at the 1 KB level, which can be ignored compared to the

storage capacity of nodes in NDN. In Kim's scheme, there is no extra storage overhead. In ELC, each core router needs to store the transaction summary and the block periodically.

3) *Communication Overhead*: Our scheme does not increase communication overhead when no attack happens. If the content poisoning attack happens, an affected router sends n_{probe} interest packets as probing packets. The probing packets and corresponding data packets only pass one hop, causing minimal costs. Besides, the additional overhead is one-time instead of persistent. Therefore, the increased communication overhead is negligible compared with the original communication overhead. In Kim's scheme, there is no extra communication overhead. In ELC, each core router needs to exchange the transaction summary with other core routers periodically.

4) *Complexity Comparison*: Firstly, we calculate the time complexity. In SLP, a router verifies the content that is hit in the caching but has not been verified previously, the complexity of which is $O(n)$, where n is the number of contents. In addition, a router inserts the information of each content into cuckoo filters and inserts each content's name into a queue probabilistically, the complexity of which is $O(n)$. Moreover, if the attack happens, a router needs to process the feedback and perform probing, the complexity of which is $O(1)$ since the number of feedback packets and probe packets is limited and does not increase with the network input. Therefore, the total complexity is $O(n)$. In Kim's scheme, when hit in the unprotected region, the content is verified and inserted into the protected region, and another content in the protected region is moved to the unprotected region. The complexity is $O(n)$. ELC adopts Kim's scheme, incurring the same overhead. Moreover, edge routers and the edge-connected routers verify each passing content, the complexity of which is $O(n)$. Besides, core routers periodically exchange information and reach consensus, the complexity of which is $O(n_{prd})$, where n_{prd} is the number of periods. Since n_{prd} is far smaller than n , the total complexity is $O(n)$.

Secondly, we calculate the space complexity. In SLP, there is a queue storing n_{probe} content names, two cuckoo filters for passing content in a MI and a cuckoo filter for verification. The space complexity is $O(1)$ since the allocated space is constant. In Kim's scheme, the space complexity is $O(1)$ since it does not need extra storage. ELC requires each core router to store the transaction summary and the block periodically, the complexity of which is $O(n_{prd})$.

5) *Summary*: In summary, during the whole process, in terms of computation, storage and communication overhead, SLP is slightly higher than Kim's scheme, while ELC is significantly higher than both.

B. Performance in Simulations

Here, we use three metrics to measure the security and efficiency of our scheme SLP:

- **Interest Satisfaction Ratio (ISR)**: The ratio of the number of received authentic contents to the number of sent interest packets.

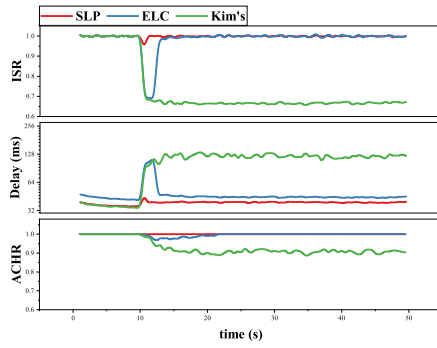


Fig. 6. Basic performance.

- Delay: The sum of the time differences between sending each interest packet and receiving corresponding content, divided by the number of received authentic contents.
- Authentic Cache Hit Ratio (ACHR): The ratio of the number of hits on authentic content in the caching to the total number of hits on all content in the caching.

1) *Basic Performance*: We test SLP, ELC and Kim's scheme in a typology of 100 nodes. The simulation lasts 50 seconds. 10 routers are randomly chosen as attackers and start the attack at 10 s with the attack power of 1, which represents the probability of modifying content. Fig. 6 shows the average ISR and delay of all users, and the average ACHR of all routers with cache hits. Before the attack, the ISR and ACHR of the three schemes are the same, while the delay of ELC is higher due to its larger computation overhead. When the attack happens, there is a slight decrease in SLP's ISR, while ELC and Kim's scheme experience a more significant decline in ISR. Besides, SLP's ISR recovers rapidly as it can quickly identify attackers. ELC's ISR recovers slowly as it relies on blockchain to find attackers, and Kim's ISR does not recover at all. Similarly, SLP and ELC exhibit different recovery speeds in delay, with SLP recovering faster than ELC, while Kim's ISR does not recover. SLP maintains an ACHR of 1 because it swiftly eliminates the attack and clears the contaminated caching. ELC experiences a slight decrease in ACHR during the attack but recovers, while Kim's exhibits a significant and unrecoverable drop in ACHR.

2) *Scenarios of Different Attack Powers*: We change the attack power from 0.001 to 1 and test the performance of three schemes, as shown in Fig. 7. The bars represent the average value within 0.5 s after the attack happens, and the lines represent the average value within 10 s after the attack happens. In Fig. 7(a), SLP's ISR in 0.5 s is close to 1, while both ELC and Kim's scheme exhibit a significant decrease in ISR in 0.5 s as the attack power increases. SLP's ISR in 10 s remains largely unchanged due to the attack being thwarted within 0.5 s. ELC's ISR in 10 s shows a slight decline as the attack power increases, since it eventually locates attackers. Kim's ISR in 10 s has a marked drop. In Fig. 7(b), SLP's delay in 0.5 s remains relatively consistent. Both ELC and Kim's scheme experience a notable increase in delay in 0.5 seconds as the attack power rises, while ELC has a higher delay due to its larger computation overhead. SLP's delay in 10 s remains largely unchanged since the attack is blocked within 0.5 s. ELC's delay in 10 s increases slowly with rising attack power

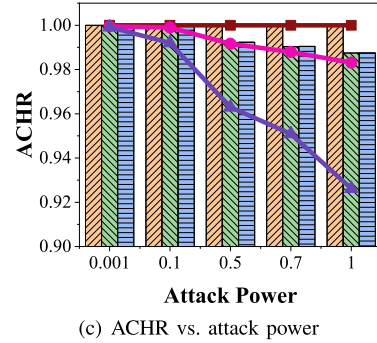
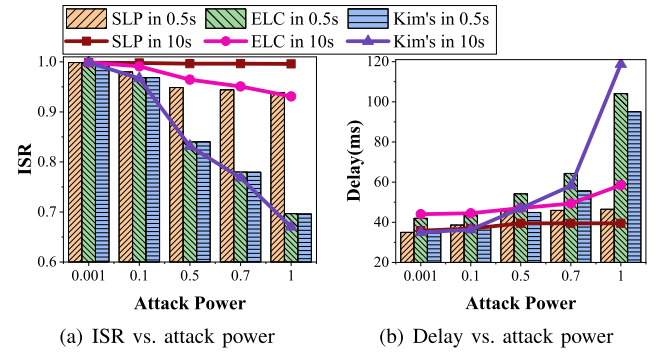


Fig. 7. Performance under scenarios of different attack powers.

since it finds attackers finally, and Kim's delay in 10 s shows a significant increase. In Fig. 7(c), SLP's ACHR in 0.5 s is always 1. Both ELC and Kim's scheme have a negligible decrease in ACHR in 0.5 s as the attack power increases since the duration of 0.5 seconds is insufficient to cache much fake content. SLP's ACHR in 10 s does not change basically since the attack is blocked within 0.5 s. ELC exhibits a small decrease in ACHR in 10 s as the attack power increases since it can find attackers finally, while Kim's scheme has a notable decrease in ACHR in 10 s.

3) *Scenarios of Different Numbers of Attackers*: We vary the number of attackers from 10 to 35, and test the performance of three schemes, as shown in Fig. 8. The attack power is 1. The bars represent the average value within 0.5 s after the attacks happens, and the lines represent the average value within 10 s after the attacks happens. In Fig. 8(a), all schemes' ISR in 0.5 s decrease notably as the number of attackers increases, but SLP's ISR is comparatively higher. SLP's ISR in 10 s doesn't decline significantly with the number of attackers due to its rapid identification of attackers. Conversely, the ISR of ELC and Kim's scheme in 10 s decreases significantly, with ELC exhibiting a higher ISR compared to Kim's due to its capability to identify attackers eventually. Fig. 8(b) shows a clear increase in the delay in 0.5 seconds of all schemes as the number of attackers rises, with ELC the highest due to its greater computation overhead and SLP the lowest since it can identify a part of attackers in 0.5 s. SLP's delay in 10 s shows a slight increase as the attack is blocked quickly. The delay of ELC and Kim's scheme in 10 s increases significantly with the number of attackers but ELC's delay is lower as it can find attackers finally. In Fig. 8(c), the ACHR of all schemes in 0.5 s decreases pretty slightly with the increasing number of attackers, but the ACHR of SLP in 0.5 s is much higher since

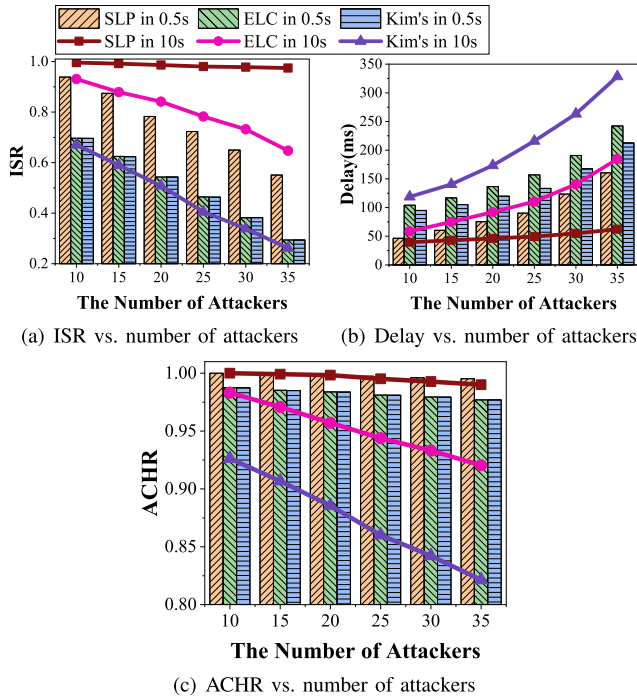


Fig. 8. Performance under scenarios of different numbers of attackers.

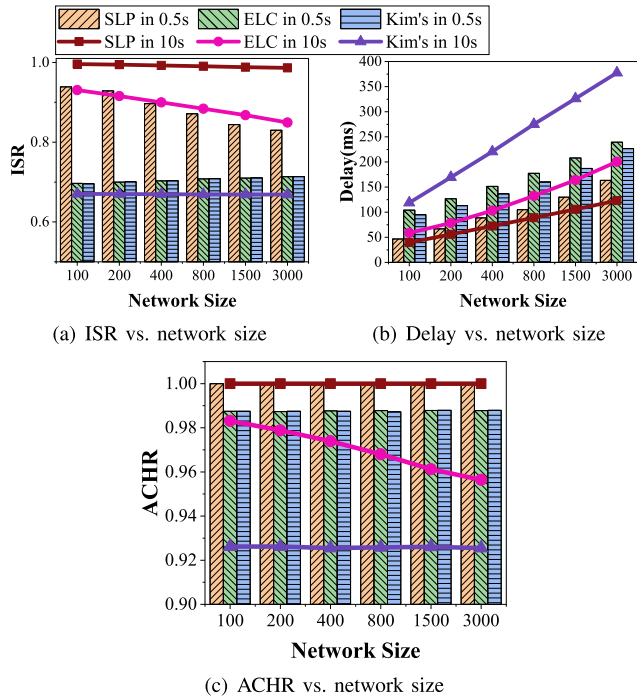


Fig. 9. Performance under scenarios of different network sizes.

it can identify a part of attackers in 0.5 s. SLP's ACHR in 10 s remains relatively stable since the attack is blocked quickly. The ACHR of ELC and Kim's scheme in 10 s decreases noticeably with the number of attackers but ELC maintains a higher ACHR since it can find attackers finally.

4) *Scenarios of Different Network Sizes*: We adjust the network size from 100 to 3000, and test the performance of three schemes, as depicted in Fig. 9. The bars represent the average value within 0.5 s after the attacks happens, and the

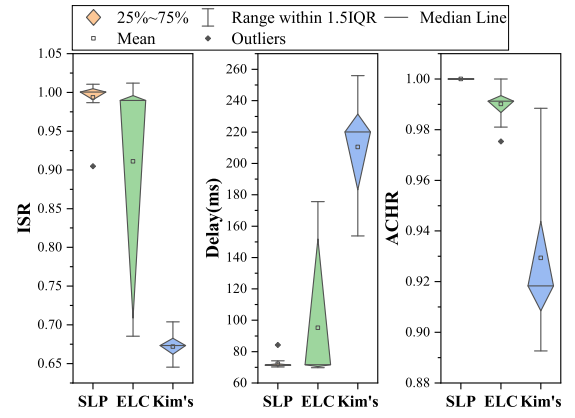


Fig. 10. Performance under the scenario with varying delays.

TABLE IV
PERFORMANCE UNDER COLLUSION ATTACK

Scheme	ISR in 10s	delay in 10s	ACHR in 10s
SLP	0.98	42.05 ms	1.00
Kim's	0.67	118.81 ms	0.92
ELC	0.93	58.08 ms	0.98

lines represent the average value within 10 s after the attacks happens. In Fig. 9(a), SLP's ISR in 0.5 s decreases with the network size since it costs more time to identify attackers in a larger network, and the ISR of ELC and Kim's scheme in 0.5 s increases slightly with the network size due to the longer distance between attackers and users. However, SLP's ISR is always the highest since it identifies a part of attackers in 0.5 s. SLP's ISR in 10 s doesn't decrease obviously with the network size since it can identify attackers quickly. ELC's ISR in 10 s diminishes notably as it takes more time to find attackers. Kim's ISR in 10 s increases not obviously due to the longer distance between attackers and users. In Fig. 9(b), the delay of all schemes in 0.5 s increases obviously with the network size due to the longer transmission distance of content, with ELC exhibiting the highest delay due to its greater computation overhead and SLP demonstrating the lowest delay since it can identify a proportion of attackers in 0.5 s. The delay of all schemes in 10 s increase obviously with the network size due to the longer transmission distance of content, with Kim's delay the highest since it cannot find attackers and SLP's delay the lowest since it can identify a part of attackers quickly. In Fig. 9(c), the ACHR of all schemes in 0.5 s exhibits negligible changes with the network size. SLP's ACHR in 10 s is always 1 since the attack is blocked quickly. ELC's ACHR in 10 s decreases obviously with the network size since it costs more time to find attackers. Kim's ACHR keeps the low level since attackers cannot be found.

5) *Scenario With Varying Delays*: To reflect the influence of varying delays in realistic networks, we make the link delay vary over time and follow the uniform distribution between 5 ms and 15 ms, and then test the performance. Fig. 10 shows the values of ISR, Delay and ACHR within 10 seconds after the attack. In the varying delay scenario, benign packets are delayed unexpectedly, causing SLP to detect attackers later than in the fixed delay scenario. However, SLP still outperforms other schemes in these three metrics.

6) *Scenario Under Collusion Attack*: To reflect the influence of collusion attack, we conduct experiments where multiple attackers are placed on the same probing path and collude to try to help each other evade detection, as shown in Table IV. Though the collusion leads to longer detection time, SLP still outperforms in terms of ISR, delay and ACHR in 10 seconds after attack, since SLP can quickly identify the closest malicious router to users.

VIII. CONCLUSION

In this paper, we proposed a secure and lightweight scheme against content poisoning attacks based on probing. We explored the challenges of accurately identifying attackers and designed a proactive probing protocol to address them. In addition, a pair of hypothesis testing is combined to analyze probing results, excluding the interference of transmission errors and flexible attacking behaviors. Moreover, users' feedback is utilized to avoid unnecessary probing costs, with its reliability guaranteed by an efficient feedback processing algorithm. Finally, through analysis on security and performance, we proved that our scheme is secure and lightweight enough.

REFERENCES

- [1] V. Jacobson, D. K. Smetters, J. D. Thornton, M. F. Plass, N. H. Briggs, and R. L. Braynard, "Networking named content," in *Proc. Int. Conf. Emerg. Netw. Exp. Technol.*, 2009, pp. 1–12.
- [2] R. Tourani, S. Misra, T. Mick, and G. Panwar, "Security, privacy, and access control in information-centric networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 1, pp. 566–600, 1st Quart., 2018.
- [3] E. G. AbdAllah, H. S. Hassanein, and M. Zulkernine, "A survey of security attacks in information-centric networking," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 3, pp. 1441–1454, 3rd Quart., 2015.
- [4] K. Xue, P. He, J. Yang, Q. Xia, and D. S. L. Wei, "SCD2: Secure content delivery and deduplication with multiple content providers in information centric networking," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1849–1864, Aug. 2022.
- [5] Z. Zhang et al., "An overview of security support in named data networking," *IEEE Commun. Mag.*, vol. 56, no. 11, pp. 62–68, Nov. 2018.
- [6] K. Xue et al., "A secure, efficient, and accountable edge-based access control framework for information centric networks," *IEEE/ACM Trans. Netw.*, vol. 27, no. 3, pp. 1220–1233, Jun. 2019.
- [7] Q. Li, R. Sandhu, X. Zhang, and M. Xu, "Mandatory content access control for privacy protection in information centric networks," *IEEE Trans. Depend. Secure Comput.*, vol. 14, no. 5, pp. 494–506, Sep. 2017.
- [8] P. Gasti, G. Tsudik, E. Uzun, and L. Zhang, "DoS and DDoS in named data networking," in *Proc. 22nd Int. Conf. Comput. Commun. Netw. (ICCCN)*, Jul. 2013, pp. 1–7.
- [9] P. He, K. Xue, J. Yang, Q. Xia, J. Liu, and D. S. L. Wei, "FASE: Fine-grained accountable and space-efficient access control for multimedia content with in-network caching," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 4, pp. 4462–4475, Dec. 2021.
- [10] M. Baugher, B. Davie, A. Narayanan, and D. Oran, "Self-verifying names for read-only named data," in *Proc. IEEE INFOCOM Workshops*, Mar. 2012, pp. 274–279.
- [11] C. Ghali, G. Tsudik, and E. Uzun, "Network-layer trust in named-data networking," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 5, pp. 12–19, Oct. 2014.
- [12] Q. Li, X. Zhang, Q. Zheng, R. Sandhu, and X. Fu, "LIVE: Lightweight integrity verification and content access control for named data networking," *IEEE Trans. Inf. Forensics Security*, vol. 10, no. 2, pp. 308–320, Feb. 2015.
- [13] D. Kim, J. Bi, A. V. Vasilakos, and I. Yeom, "Security of cached content in NDN," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 12, pp. 2933–2944, Dec. 2017.
- [14] R. Li, H. Asaeda, and J. Wu, "DCAuth: Data-centric authentication for secure in-network big-data retrieval," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 1, pp. 15–27, Jan. 2020.
- [15] C. Ghali, G. Tsudik, and E. Uzun, "In content we trust: Network-layer trust in content-centric networking," *IEEE/ACM Trans. Netw.*, vol. 27, no. 5, pp. 1787–1800, Oct. 2019.
- [16] H. Huang, Y. Wu, F. Xiao, and R. Malekian, "An efficient signature scheme based on mobile edge computing in the NDN-IoT environment," *IEEE Trans. Comput. Social Syst.*, vol. 8, no. 5, pp. 1108–1120, Oct. 2021.
- [17] Aroosa, S. S. Ullah, S. Hussain, R. Alroobaea, and I. Ali, "Securing NDN-based Internet of Health Things through cost-effective signcryption scheme," *Wireless Commun. Mobile Comput.*, vol. 2021, no. 1, pp. 1–13, Jan. 2021.
- [18] K. Xue et al., "CSEVP: A collaborative, secure, and efficient content validation protection framework for information centric networking," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 2, pp. 1761–1775, Jun. 2022.
- [19] K. Ding, J. Yang, J. Han, B. Wang, R. Li, and K. Xue, "RPBV: Reputation-based probabilistic batch verification scheme for named data networking," in *Proc. IEEE/ACM 31st Int. Symp. Quality Service (IWQoS)*, Jun. 2023, pp. 1–10.
- [20] S. DiBenedetto and C. Papadopoulos, "Mitigating poisoned content with forwarding strategy," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Jun. 2016, pp. 164–169.
- [21] C. Ghali, G. Tsudik, and C. A. Wood, "Mitigating on-path adversaries in content-centric networks," in *Proc. IEEE 42nd Conf. Local Comput. Netw. (LCN)*, Oct. 2017, pp. 27–34.
- [22] W. Cui, Y. Li, Y. Xin, and C. Liu, "Feedback-based content poisoning mitigation in named data networking," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2018, pp. 759–765.
- [23] Z. Rezaei, J. Wang, and H. Oh, "A trust-based method for mitigating cache poisoning in name data networking," *J. Netw. Comput. Appl.*, vol. 104, pp. 117–132, Feb. 2018.
- [24] T. Nguyen et al., "A security monitoring plane for named data networking deployment," *IEEE Commun. Mag.*, vol. 56, no. 11, pp. 88–94, Nov. 2018.
- [25] P. Yue, R. Li, and B. Pang, "A light-weight mitigation scheme on the mole content poisoning attack in NDN," in *Proc. 21st Asia-Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sep. 2020, pp. 132–137.
- [26] B. K. Saha and S. Misra, "Mitigating NDN-based fake content dissemination in opportunistic mobile networks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 6, pp. 1375–1386, Jun. 2020.
- [27] P. Kar, S. Misra, A. K. Mandal, and H. Wang, "SOS: NDN based service-oriented game-theoretic efficient security scheme for IoT networks," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 3, pp. 3197–3208, Sep. 2021.
- [28] M. Anisetti, C. A. Ardagna, F. Berto, and E. Damiani, "A security certification scheme for information-centric networks," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 3, pp. 2397–2408, Sep. 2022.
- [29] D. Qu, G. Lv, S. Qu, H. Shen, Y. Yang, and Z. Heng, "An effective and lightweight countermeasure scheme to multiple network attacks in NDN," *IEEE/ACM Trans. Netw.*, vol. 30, no. 2, pp. 515–528, Apr. 2022.
- [30] T. Yu, X. Ma, H. Xie, Y. Kocaogullar, and L. Zhang, "A new API in support of NDN trust schema," in *Proc. 10th ACM Conf. Inf.-Centric Netw.*, Oct. 2023, pp. 46–54.
- [31] Q. Li, P. P. C. Lee, P. Zhang, P. Su, L. He, and K. Ren, "Capability-based security enforcement in named data networking," *IEEE/ACM Trans. Netw.*, vol. 25, no. 5, pp. 2719–2730, Oct. 2017.
- [32] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than Bloom," in *Proc. 10th ACM Int. Conf. Emerg. Netw. Exp. Technol.*, Dec. 2014, pp. 75–88.
- [33] M. T. Vu, T. J. Oechtering, and M. Skoglund, "Hypothesis testing and identification systems," *IEEE Trans. Inf. Theory*, vol. 67, no. 6, pp. 3765–3780, Jun. 2021.
- [34] M. Zhang, H. Luo, and H. Zhang, "A survey of caching mechanisms in information-centric networking," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 3, pp. 1473–1499, 3rd Quart., 2015.
- [35] Z. Zhang, C.-H. Lung, X. Wei, M. Chen, S. Chatterjee, and Z. Zhang, "In-network caching for ICN-based IoT (ICN-IoT): A comprehensive survey," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14595–14620, Jun. 2023.
- [36] X. Wang, J. Lv, M. Huang, K. Li, J. Li, and K. Ren, "Energy-efficient ICN routing mechanism with QoS support," *Comput. Netw.*, vol. 131, pp. 38–51, Feb. 2018.
- [37] C. Griner, S. Schmid, and C. Avin, "CacheNet: Leveraging the principle of locality in reconfigurable network design," *Comput. Netw.*, vol. 204, Feb. 2022, Art. no. 108648.

- [38] I. Mirzadeh, M. Sayad Haghighi, and A. Jolfaei, "Filtering malicious messages by trust-aware cognitive routing in vehicular ad hoc networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 1134–1143, Jan. 2023.
- [39] P. Zhang et al., "Network-wide forwarding anomaly detection and localization in software defined networks," *IEEE/ACM Trans. Netw.*, vol. 29, no. 1, pp. 332–345, Feb. 2021.
- [40] S. Mastorakis, A. Afanasyev, I. Moiseenko, and L. Zhang, "ndnSIM 2: An updated NDN simulator for NS-3," Univ. California Los Angeles, Los Angeles, CA, USA, Tech. Rep. NDN-0028, 2016. Accessed: Aug. 2024. [Online]. Available: <https://named-data.net/wp-content/uploads/2016/11/ndn-0028-2-ndnsim-v2.pdf>



Jiangping Han (Member, IEEE) received the bachelor's and Ph.D. degrees from the Department of Electronic Engineering and Information Science (EEIS), USTC, in 2016 and 2021, respectively. From November 2019 to October 2021, she was a Visiting Scholar with the School of Computing, Informatics, and Decision Systems Engineering, Arizona State University. She was a Post-Doctoral Researcher with the School of Cyber Science and Technology, USTC, in 2021 and 2023, where she is currently an Associate Researcher with the School of Cyber Science and Technology. Her research interests include future internet architecture design and transmission optimization.



Kunpeng Ding received the B.S. degree in information security from the School of Cyber Science and Technology, University of Science and Technology of China (USTC), in 2022, where he is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology. His research interests include network security protocol design and analysis.



Jian Li (Senior Member, IEEE) received the bachelor's degree from the Department of Electronics and Information Engineering, Anhui University, in 2015, and the Ph.D. degree from the Department of Electronic Engineering and Information Science (EEIS), University of Science and Technology of China (USTC), in 2020. From November 2019 to November 2020, he was a Visiting Scholar with the Department of Electronic and Computer Engineering, University of Florida. From December 2020 to December 2022, he was a Post-Doctoral Researcher with the School of Cyber Science and Technology, USTC, where he is currently an Associate Researcher with the School of Cyber Science and Technology. His research interests include quantum networking, wireless networks, and next-generation internet architecture. He serves as an Editor for *China Communications*.



Jiayu Yang (Member, IEEE) received the bachelor's and Ph.D. degrees from the School of Cyber Security, University of Science and Technology of China (USTC) in 2019 and 2024, respectively. Her research interests include future internet architecture design, transmission optimization, and network security.



Qibin Sun (Fellow, IEEE) received the Ph.D. degree from the Department of Electronic Engineering and Information Science (EEIS), University of Science and Technology of China (USTC), in 1997. He is currently a Professor with the School of Cyber Science and Technology, USTC. He has published more than 120 papers in international journals and conferences. His research interests include multimedia security and network intelligence and security.



Kaiping Xue (Senior Member, IEEE) received the bachelor's degree from the Department of Information Security, University of Science and Technology of China (USTC), in 2003, and the Ph.D. degree from the Department of Electronic Engineering and Information Science (EEIS), USTC, in 2007. From May 2012 to May 2013, he was a Post-Doctoral Researcher with the Department of Electrical and Computer Engineering, University of Florida. Currently, he is a Professor with the School of Cyber Science and Technology, USTC, where he is also the

Director of Network and Information Center. His research interests include next-generation Internet architecture design, transmission optimization, and network security. He is an IET Fellow. His work won best paper awards in IEEE MSN 2017 and IEEE HotICN 2019, the Best Paper Honorable Mention in ACM CCS 2022, the Best Paper Runner-Up Award in IEEE MASS 2018, and the best track paper in MSN 2020. He serves on the editorial board for several journals, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY (IEEE TIFS), IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING (IEEE TDSC), IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS (IEEE TWC), and IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT (IEEE TNSM). He has served as a Guest Editor for many reputed journals/magazines, including IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS (IEEE JSAC), *IEEE Communications Magazine*, and *IEEE Network*.



Jun Lu received the bachelor's degree from Southeast University in 1985 and the master's degree from the Department of Electronic Engineering and Information Science (EEIS), University of Science and Technology of China (USTC), in 1988. Currently, he is a Professor with the School of Cyber Science and Technology and the Department of EEIS, USTC. He is also the President of Jiaxing University. His research interests include theoretical research and system development in the field of integrated electronic information systems, networks,

and information security. He is an Academician of Chinese Academy of Engineering (CAE).